

Explications approfondies : ColorRoom (E2)

Pour chaque notion : ● **version complexe** (pro, pour impressionner) + ● **version simple** (claire, si on te demande de reformuler) + un vrai bloc de code du dépôt [NewGenesisAgency/color_room](https://github.com/NewGenesisAgency/color_room).

Conseil d'oral : annonce d'abord la version complexe ; si le jury fronce les sourcils, enchaîne « autrement dit... » et donne la version simple, puis **montre le code**.

Sommaire : 1. Variable transactionnelle (ACID) · 2. Variable atomique / sémaphore · 3. Variable volatile · 4. Persistance, WAL & migrations · 5. Hachage PBKDF2 · 6. Sessions & cookies · 7. Requêtes préparées · 8. IA minimax + alpha-bêta · 9. Réseau de neurones vs minimax · 10. Multijoueur (polling) · 11. Concurrency matérielle (Promise.all + Abort) · 12. React : déclaratif & état réactif.

1. Variable transactionnelle (ACID)

● **Version complexe** : une transaction encapsule plusieurs écritures dans une unité **ACID**.

`db.transaction()` de better-sqlite3 émet un `BEGIN`, exécute le corps, puis `COMMIT` ; si une exception est levée, il effectue un `ROLLBACK` automatique. J'en ai besoin pour l'inscription, qui touche **deux tables** (`crg_users` puis `crg_class_members`) : c'est l'**atomicité** (le « A » d'ACID) qui garantit l'absence d'état intermédiaire incohérent.

● **Version simple** : c'est un « tout ou rien ». Soit l'utilisateur **et** son inscription à la classe sont créés, soit **rien** n'est créé. Jamais de compte « à moitié fait ».

Exemple concret `*(app/app/api/auth/register/route.ts)*` :

```
const id    = randomBytes(16).toString('hex');
const hash = hashPassword(password);

// db.transaction() = BEGIN ... COMMIT (ROLLBACK auto si une ligne échoue)
const insertAll = db.transaction(() => {
  db.prepare(
    "INSERT INTO crg_users (id, name, user_type, password_hash, avatar_color, avatar_icon) \
    VALUES (?, ?, 'apprenant', ?, ?, ?)"
  ).run(id, usernameClean, hash, color, icon);

  if (classCode?.trim()) {
    // jonction de classe optionnelle
    const classRow = db.prepare("SELECT id FROM crg_classes WHERE code = ?")
      .get(classCode.trim().toUpperCase() as { id: string } | undefined);
    if (classRow) {
      db.prepare("INSERT OR IGNORE INTO crg_class_members (id, class_id, user_id) VALUES (?, \
      ?, ?)")
        .run(randomBytes(16).toString('hex'), classRow.id, id);
    }
  }
});

insertAll(); // exécution atomique : tout, ou rien
```

2. Variable atomique / sémaphore matériel

● **Version complexe** : `hwInFlight` est un **compteur partagé** qui borne le nombre d'appels concurrents vers `supervision.exe` (ressource quasi-série). C'est un **sémaphore** à `HW_CONCURRENCY = 2` jetons. Sa

lecture-modification (`if (hwInFlight < N) hwInFlight++`) est **atomique de fait** car la boucle d'événements de Node est **mono-thread** : aucune préemption ne peut s'intercaler entre le test et l'incrément. Au-delà des jetons, les requêtes s'enregistrent dans une **file de Promise** réveillée à la libération d'un slot.

● **Version simple** : c'est un videur de boîte de nuit. Le matériel n'accepte que **2 personnes à la fois** ; `hwInFlight` compte combien sont déjà entrées. Les suivants **font la queue** et entrent dès qu'une place se libère.

Exemple concret `*(app/app/api/supervision/batch/route.ts)*` :

```
const HW_CONCURRENCY = 2;           // supervision.exe est quasi-série
let hwInFlight = 0;                  // compteur "atomique" (slots occupés)
const hwWaiters: Waiter[] = [];     // file d'attente de promesses

async function acquireHwSlot(): Promise<boolean> {
  if (hwInFlight < HW_CONCURRENCY) { // place libre → j'entre
    hwInFlight++;
    return true;
  }
  return new Promise<boolean>((resolve) => { // sinon → je fais la queue
    hwWaiters.push({ resolve });
  });
}

function releaseHwSlot() {
  hwInFlight = Math.max(0, hwInFlight - 1); // je sors
  drainWaiters();                          // je réveille le suivant
}

function drainWaiters() {
  while (hwWaiters.length > 0 && hwInFlight < HW_CONCURRENCY) {
    hwInFlight++;
    hwWaiters.shift()!.resolve(true);      // le 1er de la file entre
  }
}
```

3. Variable volatile (en mémoire)

● **Version complexe** : une variable **volatile** réside en mémoire vive pour la durée du composant (état runtime). En React, `useRef` fournit une **référence mutable persistante entre les rendus mais qui ne déclenche pas de re-rendu** ; `useState` fournit un état **volatile réactif** (sa mutation planifie une réconciliation). Par opposition, l'état **persistant** est sérialisé en SQLite. J'utilise du volatile pour l'état haute fréquence d'un jeu (combo, dalle active, chrono), et je ne **persiste** que le résultat.

● **Version simple** : c'est une note sur un Post-it : pratique pendant la partie, **jetée à la fin**. Si je recharge la page, c'est perdu. Seul le **score final** est écrit « dans le cahier » (la base).

Exemple concret `*(app/app/_components/GameColorSpeed.tsx)*` :

```
// Volatile RÉACTIF : changer la valeur ré-affiche l'écran
const [phase, setPhase] = useState<Phase>('ready');
const [score, setScore] = useState(0);
const [timeLeft, setTimeLeft] = useState(cfg.duration);

// Volatile PUR : vit en mémoire, NE déclenche PAS de re-rendu (très rapide)
const comboRef = useRef(0); // combo en cours
const lightRef = useRef(0); // dalle actuellement allumée
const tileStartRef = useRef(0); // instant d'allumage (pour le scoring)

// ... à la fin seulement, on PERSISTE le score en base :
// db.prepare("INSERT INTO crg_scores (user_id, game_id, score) VALUES (?, ?, ?)").run(uid,
// gid, score);
```

4. Persistance, WAL & migrations

● **Version complexe** : la couche données est **SQLite** via **better-sqlite3** (API synchrone). Au démarrage, **migrate()** applique des **PRAGMA** : **WAL** (*Write-Ahead Logging*) autorise des lectures concurrentes pendant une écriture, **busy_timeout** borne l'attente avant **SQLITE_BUSY**, **foreign_keys = ON** active l'intégrité référentielle. Les tables sont créées en **CREATE TABLE IF NOT EXISTS** : migrations **idempotentes** (une base neuve se crée seule, une base existante n'est pas cassée).

● **Version simple** : la base, c'est **un fichier**. Au lancement, le code crée les tables **si elles n'existent pas** (donc on peut relancer sans rien casser), et active un mode (WAL) qui permet de **lire et écrire en même temps** sans blocage.

Exemple concret `*(app/lib/db/index.ts)*` :

```
function migrate(db: Database.Database) {
  db.pragma('journal_mode = WAL'); // lectures pendant une écriture
  db.pragma('synchronous = NORMAL');
  db.pragma('busy_timeout = 5000'); // réessaie 5 s au lieu d'échouer
  db.pragma('foreign_keys = ON'); // intégrité des relations

  db.exec(
    "CREATE TABLE IF NOT EXISTS crg_mp_sessions (\
      id TEXT PRIMARY KEY, status TEXT NOT NULL, game_id TEXT NOT NULL, \
      state_json TEXT NOT NULL, updated_at TEXT NOT NULL DEFAULT (datetime('now')));"
  );
  db.exec(
    "CREATE TABLE IF NOT EXISTS crg_mp_players (\
      id TEXT PRIMARY KEY, session_id TEXT NOT NULL, seat INTEGER NOT NULL, \
      FOREIGN KEY(session_id) REFERENCES crg_mp_sessions(id));"
  );
}
```

5. Hachage des mots de passe (PBKDF2)

● **Version complexe** : les mots de passe sont stockés sous forme de **dérivation de clé** PBKDF2-HMAC-SHA512, **100 000 itérations**, **sel** aléatoire de 16 octets, clé dérivée de 64 octets, au format **sel:hash**. Le coût itératif ralentit le brute-force ; le sel unique neutralise les **rainbow tables**. La vérification **recalcule** la dérivation avec le même sel et compare.

● **Version simple** : je ne garde **jamais** le mot de passe. J'en garde une **empreinte** impossible à inverser, calculée volontairement **lentement** (pour décourager les attaques) et avec un grain de sel **unique** par compte. Pour vérifier, je recalculer l'empreinte et je compare.

Exemple concret *(app/lib/auth.ts)* :

```
export function hashPassword(password: string): string {
  const salt = randomBytes(16).toString('hex'); // sel unique
  const hash = pbkdf2Sync(password, salt, 100_000, 64, 'sha512') // 100 000 tours
    .toString('hex');
  return `${salt}:${hash}`; // on stocke
  sel:hash
}

export function verifyPassword(password: string, stored: string): boolean {
  const [salt, hash] = stored.split(':');
  if (!salt || !hash) return false;
  const verify = pbkdf2Sync(password, salt, 100_000, 64, 'sha512').toString('hex');
  return verify === hash; // recalculer +
  compare
}
```

6. Sessions & cookies

● **Version complexe** : à la connexion, je génère un **jeton opaque** (`randomBytes(32)`) stocké en base (`crg_sessions`) avec une expiration à **30 jours**, puis posé dans un cookie **HttpOnly** (inaccessible au JavaScript → anti-XSS) et **SameSite=lax** (non envoyé en requête cross-site → anti-CSRF). Les sessions expirées de l'utilisateur sont **purgées** à la création. Un renouvellement glissant prolonge la validité à l'usage.

● **Version simple** : quand tu te connectes, je te donne un **bracelet** (un jeton aléatoire) que je note dans la base. Ton navigateur le garde dans un cookie **que le JavaScript ne peut pas lire** (sécurité). Tant que le bracelet est valable (30 jours), tu restes connecté.

Exemple concret *(app/lib/auth.ts + route de login)* :

```
export function createSession(userId: string): string {
  const db = getDb();
  const token = randomBytes(32).toString('hex'); // jeton opaque
  const expiresAt = new Date(Date.now() + 30*24*3600*1000).toISOString(); // +30 jours
  db.prepare("DELETE FROM crg_sessions WHERE user_id = ? AND expires_at < datetime('now')")
    .run(userId); // purge l'expiré
  db.prepare("INSERT INTO crg_sessions (id, user_id, token, expires_at) VALUES (?, ?, ?, ?)")
    .run(randomBytes(16).toString('hex'), userId, token, expiresAt);
  return token;
}

// Côté route POST /api/auth/login : on pose le cookie sécurisé
res.cookies.set('crg_session', token, {
  httpOnly: true, // inaccessible au JS → anti-XSS
  sameSite: 'lax', // pas de cross-site → anti-CSRF
  maxAge: 30 * 24 * 3600,
  path: '/',
});
```

7. Requêtes préparées (anti-injection SQL)

● **Version complexe** : toutes les requêtes passent par `db.prepare(...)` avec des **paramètres liés** (`?`). Les valeurs ne sont jamais concaténées dans la chaîne SQL : elles sont transmises séparément au moteur, qui les traite comme des **données** et non comme du **code SQL** → **injection SQL** impossible. Bonus : la requête préparée est **compilée une fois** puis réutilisée (plus rapide).

● **Version simple** : je ne « colle » jamais ce que tape l'utilisateur dans la requête. Je mets un **trou** `?` et je donne la valeur à part. Comme ça, même si quelqu'un tape du SQL méchant dans son pseudo, c'est traité comme **du texte**, pas comme une commande.

Exemple concret `*(app/app/api/auth/register/route.ts)*` :

```
// ✅ paramètre lié : la valeur est traitée comme une donnée
const existing = db.prepare('SELECT id FROM crg_users WHERE name = ? COLLATE NOCASE')
  .get(usernameClean);

// ❌ ce qu'on ne fait JAMAIS (vulnérable à l'injection) :
// db.prepare(`SELECT id FROM crg_users WHERE name = '${usernameClean}'`).get();
```

8. IA Puissance 4 : minimax + élagage alpha-bêta

● **Version complexe** : l'IA explore l'**arbre de jeu** par **minimax** : aux nœuds MAX (l'IA) on maximise, aux nœuds MIN (l'adversaire) on minimise, en supposant un adversaire optimal. La récursion est bornée par une **profondeur**. Aux feuilles, une **fonction d'évaluation heuristique** somme le score de toutes les **fenêtres de 4** (`scoreWindow`). L'**élagage alpha-bêta** maintient les bornes (alpha, beta) et **coupe** les sous-arbres ne pouvant influencer la décision ; un **ordonnancement des coups** (centre d'abord) maximise les coupures.

● **Version simple** : l'IA **joue dans sa tête** plusieurs coups à l'avance : « si je joue ici, l'adversaire jouera là, alors moi... ». Elle garde le coup qui lui donne la meilleure position en supposant que l'adversaire joue parfaitement. L'**alpha-bêta** lui évite d'explorer les coups qui sont déjà clairement mauvais → elle réfléchit **plus loin, plus vite**.

Exemple concret `*(app/app/_components/GamePuissance4.tsx)*` :

```
// (1) Évaluation : note une fenêtre de 4 cases, du point de vue de l'IA
function scoreWindow(me: number, opp: number): number {
  if (me > 0 && opp > 0) return 0; // fenêtre mixte = inexploitable
  if (me === 4) return WIN_SCORE; // victoire = 1 000 000
  if (me === 3) return 130; // mon alignement de 3
  if (opp === 3) return -170; // menace adverse : DÉFENSE > attaque
  return 0;
}

// (2) Minimax + alpha-bêta
function minimax(grid, depth, alpha, beta, maximizing): number {
  // ... arrêt : depth 0 ou plateau plein → return evaluateBoard(grid)
  if (maximizing) { // tour de l'IA → MAX
    let value = -Infinity;
    for (const c of orderColumns(valid)) { // centre d'abord
      const d = dropAt(grid, c, AI)!;
      const v = winsAt(d.grid, d.row, c, AI) ? WIN_SCORE + depth
        : minimax(d.grid, depth - 1, alpha, beta, false);
      value = Math.max(value, v);
      alpha = Math.max(alpha, value);
      if (alpha >= beta) break; // ✂ coupure bêta
    }
    return value;
  }
  // sinon : tour adverse → MIN, symétrique, avec "✂ coupure alpha"
}
}
```

Niveaux = profondeur : `novice {depth:1}` ... `legendaire {depth:12}` .

9. Réseau de neurones vs minimax (ce que je n'utilise PAS, et pourquoi)

● **Version complexe** : un **réseau de neurones** est un approximateur de fonction paramétré par des **poids** appris par **descente de gradient / rétropropagation** sur un corpus. Pour Puissance 4, l'approche neuronale (type AlphaZero) exige un **entraînement** coûteux et des **données** massives ; elle produit une politique implicite, peu **explicable**. J'ai préféré une approche **symbolique** (minimax), déterministe, sans données, **explicable ligne à ligne**, et adaptée à un Raspberry Pi **hors-ligne**.

● **Version simple** : un réseau de neurones **apprend** à force d'exemples (comme un élève qui s'entraîne sur des milliers de parties). Mon IA, elle, **ne s'entraîne pas** : elle **calcule** les coups sur le moment. C'est plus léger, instantané, et je peux **expliquer chaque décision**.

Comparaison (pseudo-code) :

```
RÉSEAU DE NEURONES (non utilisé) :
  1) entraînement (hors-ligne, lent) : poids ← apprendre(des millions de parties)
  2) en jeu : coup ← f(plateau, poids) // "intuition" apprise, boîte noire

MINIMAX (utilisé) :
  coup ← max ( sur mes coups )
           min ( sur coups adverses )
           évaluation(plateau) // calculé en direct, explicable
```

Critère	Réseau de neurones	Minimax (mon choix)
Entraînement	obligatoire, lourd	aucun
Données	des milliers de parties	zéro
Pi hors-ligne	inadapté	léger, instantané
Explicable	boîte noire	100 % lisible

10. Multijoueur : état partagé + polling

● **Version complexe** : plutôt qu'un canal **WebSocket**, l'état d'une partie est **sérialisé en JSON** et **persisté** dans `crg_mp_sessions.state_json`. Chaque client **interroge périodiquement** (`polling`) la route `/state` pour récupérer l'état courant. C'est moins « temps réel » qu'un WebSocket, mais **sans serveur d'événements**, **robuste** aux reconnexion et trivial à déployer sur le Pi. Les joueurs (`crg_mp_players`) ont un **jeton**, un **siège** et un `last_seen_at` (heartbeat de présence).

● **Version simple** : au lieu d'un « téléphone toujours décroché » (WebSocket), chaque joueur **rafraîchit** la partie toutes les ~0,7 s en demandant « quoi de neuf ? ». L'état de la partie est **stocké en base**, donc tout le monde voit la même chose, même après une coupure réseau.

Exemple concret *(schéma + principe de polling)* :

```
// Écriture de l'état partagé (1 ligne JSON par partie)
db.prepare("UPDATE crg_mp_sessions SET state_json = ?, updated_at = datetime('now') WHERE id = ?")
  .run(JSON.stringify(state), sessionId);

// Côté client : on interroge l'état régulièrement (polling)
useEffect(() => {
  const t = setInterval(async () => {
    const r = await fetch(`/api/multiplayer/state?id=${sessionId}`);
    setState(await r.json()); // met à jour l'écran
  }, 700);
  return () => clearInterval(t);
}, [sessionId]);
```

11. Concurrency matérielle : parallélisme borné + annulation

● **Version complexe** : une dalle = **32 canaux** à envoyer ; je les émet en **parallèle** via `Promise.all` (au lieu d'attendre chaque requête en série), tout en **bornant** la concurrence globale par le sémaphore (§2). Un `AbortController` partagé permet une **annulation immédiate** : lors d'un reset, on `abort()` les requêtes en vol et on vide la file, évitant l'accumulation de requêtes **stale** qui saturerait `supervision.exe`.

● **Version simple** : pour allumer une dalle, j'envoie ses 32 réglages **en même temps** (plus rapide). Et si l'utilisateur change tout d'un coup, je peux **tout annuler instantanément** pour ne pas embouteiller le matériel.

Exemple concret *(app/app/api/supervision/batch/route.ts)* :

```
// Envoi des requêtes en parallèle, avec signal d'annulation commun
const results = await Promise.all(
  allRequests.map((r) => sendOne(r, forceAbortCtrl.signal))
);

// Reset instantané : on annule tout ce qui est en vol + on vide la file
function forceReset() {
  forceAbortCtrl.abort(); // coupe les fetch en cours
  forceAbortCtrl = new AbortController(); // nouveau contrôleur
  const stale = hwWaiters.splice(0); // vide la file d'attente
  for (const w of stale) w.resolve(false); // les waiters ne lancent rien
}
```

12. React : programmation déclarative & état réactif

● **Version complexe** : l'UI est décrite de façon **déclarative** : je rends une fonction de l'**état** vers une vue (TSX). Une mutation d'état via `useState` planifie une **réconciliation** : React compare le nouvel arbre virtuel à l'ancien (**Virtual DOM**) et n'applique au DOM réel que le **diff** minimal. Le flux de données est **unidirectionnel** (état → vue), ce qui rend le rendu prévisible.

● **Version simple** : je ne dis pas « va changer ce texte » ; je dis « **voici à quoi doit ressembler l'écran selon l'état** », et React se débrouille pour ne modifier **que ce qui a changé**. Quand le score change, seul le score se re-dessine.

Exemple concret *(structure type d'un jeu)* :

```
const [score, setScore] = useState(0); // état réactif

function makeMove(col: number) {
  // ... logique ...
  setScore((s) => s + points); // déclenche un re-rendu ciblé
}

return ( // description déclarative de l'UI
  <div className="board">
    {columns.map((c) => (
      <button key={c} onClick={() => makeMove(c)} />
    ))}
    <span>Score : {score}</span> /* se met à jour tout seul */
  </div>
);
```

13. Three.js : la vue 3D temps réel de la salle

● **Version complexe** : la salle est rendue en WebGL via **Three.js**. Je monte une **Scene**, une **PerspectiveCamera** et un **WebGLRenderer** dont le `domElement` est inséré dans la page ; chaque **dalle** est un **Mesh** dont le matériau suit l'état du jeu. Le rendu tourne dans une **boucle** `requestAnimationFrame`. Au démontage du composant, je libère explicitement le contexte GPU (`forceContextLoss`) et j'annule la boucle, pour éviter une **fuite de contexte WebGL** (le navigateur en limite le nombre).

● **Version simple** : je dessine la salle en **3D dans le navigateur** ; chaque dalle est un petit carré 3D qui prend la couleur envoyée au jeu. Une boucle redessine l'image en continu, et quand on quitte la page je **fais le ménage** pour ne pas saturer la carte graphique.

Exemple concret *(app/app/_components/Room3D.tsx)* :


```

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(CAM_FOV, W / H, 0.05, 80);
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(W, H);
mount.appendChild(renderer.domElement);

const plate = new THREE.Mesh(geometry, material); // une dalle
scene.add(plate);

function loop() { // boucle de rendu
  renderer.render(scene, camera);
  raf = requestAnimationFrame(loop);
}
loop();

return () => { // nettoyage au démontage
  cancelAnimationFrame(raf);
  renderer.forceContextLoss(); // évite la fuite WebGL
};

```

14. Méthode agile, suivi client & versionnement Git

● **Version complexe** : projet conduit en **agile** : itérations courtes, livraisons incrémentales, priorisation continue du besoin via des **points réguliers avec le client M. Labayrade** (directeur du labo **BPMNP**). Côté code, workflow Git à deux branches : je développe et j'intègre sur **ux-last**, puis je **fusionne sur main** (branche stable, déployée). Une chaîne **CI/CD** (GitLab) build l'image Docker et la déploie sur le Raspberry Pi. La **documentation** (guide technique, 15 diagrammes UML, notice, README) a été **rédigée par moi**.

● **Version simple** : on a avancé **par petites étapes**, en montrant régulièrement le travail au **client** pour ajuster. Le code a deux branches : une **de travail** (**ux-last**) et une **propre** (**main**) ; quand c'est prêt, je **fusionne** l'une dans l'autre, et ça se déploie tout seul sur le Raspberry Pi. **C'est moi qui ai écrit la doc.**

Exemple concret *(workflow Git réel du projet)* :

```

# 1) je développe sur la branche d'intégration
git add -A && git commit -m "feat(ia): minimax alpha-beta + anti-piege"
git push -u origin ux-last

# 2) quand c'est stable, je fusionne sur main (branche déployée)
git checkout main
git merge ux-last --no-edit
git push -u origin main      # -> CI/CD GitLab -> build Docker -> Raspberry Pi

```

*Astuce finale : pour chaque notion, le réflexe gagnant = « **version pro** » → si besoin « **autrement dit...** » (version simple) → « **et voici dans le code** » (le bloc). C'est exactement ce que le jury attend en E6.*

</content>